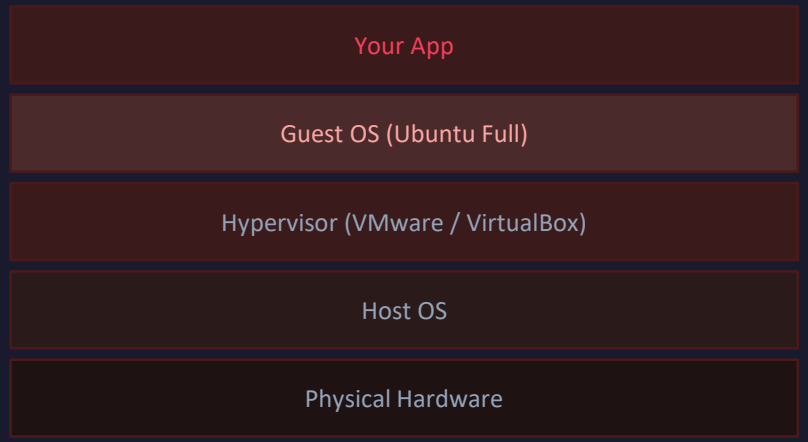
What is Docker — and Why Does it Exist?

The problem of 'it works on my machine'

The Problem: "It works on my machine!" Your laptop has Python 3.11, PyTorch 2.3, CUDA 12. Your server has Python 3.8, no PyTorch, different OS. The model crashes.

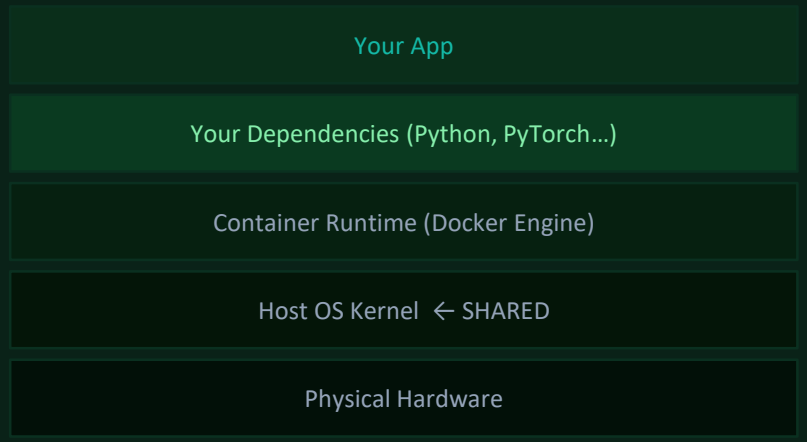
✗ Virtual Machine (VM)



- ✗ Full OS per VM (~20GB each)
- ✗ Boots in 1-2 minutes
- ✗ Heavy RAM usage (2-4GB per VM)
- ✗ Slow to start, hard to replicate

VS

✓ Docker Container



- ✓ Shares host OS kernel (~200MB image)
- ✓ Starts in under 1 second
- ✓ Lightweight (50-200MB RAM)
- ✓ Identical on laptop, server, cloud

Core Docker Concepts

Image · Container · Layer · Registry · Volume

Image

A read-only blueprint — like a recipe or class definition.

Image = Class | Container = Object (instance)

Contains: OS base layer + your code + all dependencies.
Built from a Dockerfile. Can be shared on Docker Hub.

Container

A running instance of an image. Isolated process on your machine.

Same image can run 10 containers simultaneously.

Has its own filesystem, network, process space.
Ephemeral by default — data lost when stopped (use Volumes).

Layer

Each Dockerfile instruction creates a cached layer.

Layers are shared between images — if Python 3.11 layer exists, every image using it reuses it.

COPY code = new layer. pip install = new layer.
Change code → only that layer rebuilds. Speeds up builds 10x.

Registry

Storage for Docker images. Docker Hub is the default public registry.

npm registry for Node packages, PyPI for Python — same idea for containers.

docker pull python:3.11 → downloads from Docker Hub.
You can push your own images: docker push myapp:v1.

Volume

Persistent storage mounted into a container.

A USB drive plugged into your container — survives container restarts.

Used for: database files, uploaded files, logs.
Without volumes: all data is lost when container stops.

Port Mapping

Connects host machine port to container's internal port.

-p 8080:8000 means: my laptop's :8080 → container's :8000

Container is isolated by default — ports must be explicitly opened.
Multiple containers can each have their own :8000 internally.

Essential Docker Commands

The 12 commands you'll use 95% of the time

Build & Run

`docker build -t myapp:v1 .`

Build image from Dockerfile in current dir. -t = tag name

`docker run -p 8080:8000 myapp:v1`

Run container. Map host :8080 to container :8000

`docker run -d myapp:v1`

-d = detached (background). Returns container ID

`docker run -e KEY=val myapp:v1`

-e = inject environment variable into container

Inspect & Debug

`docker ps`

List running containers (ID, name, ports, status)

`docker ps -a`

All containers including stopped ones

`docker logs myapp`

Print stdout/stderr from the container

`docker logs -f myapp`

-f = follow (live streaming logs)

`docker exec -it myapp bash`

Open interactive shell INSIDE running container

Stop & Clean

`docker stop myapp`

Gracefully stop container (SIGTERM)

`docker rm myapp`

Delete stopped container

`docker rmi myapp:v1`

Delete image from local storage

`docker system prune -a`

⚠ Delete ALL unused containers, images, networks

Compose Commands

`docker compose up --build`

Build images and start all services

`docker compose up -d`

Start all services in background (detached)

`docker compose down`

Stop and remove all containers + networks

`docker compose logs -f api`

Follow logs for specific service named 'api'

`docker compose ps`

Show status of all services in compose file

Pro tip: Use `docker exec -it <name> bash` to debug inside a container — it's like SSH but for Docker

Dockerfile Anatomy — Line by Line

Every instruction creates a cached layer

Our Project's Dockerfile

```
# ── Base image ──────────────────────────
FROM python:3.11-slim                          A

# ── Set working directory ──────────────
WORKDIR /app                                   B

# ── Install dependencies FIRST ─────────
COPY requirements.txt .                        C
RUN pip install -r requirements.txt            C

# ── Copy application code AFTER ────────
COPY . .                                       D

# ── Expose the port your app uses ──────
EXPOSE 8000                                    E

# ── Command to run on container start ──
CMD ["uvicorn","main:app","--host",            F
     "0.0.0.0","--port","8000"]
```

What each instruction does

A — `FROM python:3.11-slim`

Pull base image from Docker Hub. 'slim' = minimal OS. No GUI, no extras. 200MB vs 900MB for full image.

B — `WORKDIR /app`

All following commands run inside /app. Creates it if missing. Like cd /app permanently.

C — `COPY requirements.txt + RUN pip install`

KEY TRICK: Copy requirements BEFORE your code. If code changes but requirements don't → this layer is cached. Rebuilds are 10x faster.

D — `COPY . .`

Copy all project files into /app. Placed AFTER pip install so code changes don't invalidate the dependency cache layer.

E — `EXPOSE 8000`

Documentation: tells Docker (and humans) the container listens on port 8000. Doesn't actually open the port — that's done with -p at runtime.

F — `CMD ["uvicorn", "main:app", ...]`

Default command when container starts. Use array form (exec form) — not string. Receives OS signals correctly for graceful shutdown.

Demo 5 — Docker Compose Load Balancing

Production-identical local setup · same pattern as AWS ELB

docker-compose.yml

```yaml
version: "3"

services:
  api1:
    build: .
    ports:
      - "8000:8000"
    environment:
      - INSTANCE=api1

  api2:
    build: .
    ports:
      - "8001:8000"
    environment:
      - INSTANCE=api2

  nginx:
    image: nginx:latest
    ports:
      - "8080:80"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf
    depends_on:
      - api1
      - api2
```

How to run

```
# Build and start everything
docker compose up --build

# Test
curl http://localhost:8080/info
```

Architecture

Containerised
Each API instance in its own Docker container

Nginx as Gateway
Single entry point :8080 → routes to api1/api2

Scalable
Add api3, api4 in compose — zero downtime

Cloud-ready
Same pattern: AWS ELB, GCP LB, Azure LB

✓ This is how every cloud ML API deployment works under the hood

Docker Workflow — Build · Run · Debug · Ship

The complete cycle from code to running container

1

Write Dockerfile

2

Build Image

```
docker build
-t app:v1 .
```

3

Run Container

```
docker run
-p 8000:8000 app:v1
```

4

Debug & Fix

```
docker logs app
docker exec -it app
bash
```

5

Push & Deploy

```
docker push
myuser/app:v1
```

Layer Caching — Why Order Matters

Code changes only — Fast (~5s)

FROM ✓ cached → WORKDIR ✓ cached → pip install ✓ cached → COPY code ↻ rebuild → CMD ↻ rebuild

requirements.txt changes — Slow (~60s)

FROM ✓ cached → WORKDIR ✓ cached → pip install ↻ rebuild → COPY code ↻ rebuild → CMD ↻ rebuild